Universidad de Antioquia
Departamento de Electronica
Informatica II

Desafío 1:

Implementación de Tetris mediante manipulación de bits en C++

Andres David Guerrero Rivera
Mauricio Reyes Rosero

22 de marzo de 2026

1. Introducción

El desafío consiste en implementar una versión simplificada del videojuego Tetris en C++, con énfasis en el manejo eficiente de memoria dinámica, el uso de punteros y la representación compacta de datos mediante operaciones de bits. El tablero de juego se representa como un arreglo dinámico de bytes donde cada celda ocupa exactamente un bit, y los siete tipos de piezas clásicas (T, O, I, L, J, S, Z), se codifican como enteros de 16 bits que describen una cuadrícula 4×4, sobre la cual se realizan operaciones de rotación, desplazamiento y fijado. Una vez que una pieza se fija al tablero, este verifica y elimina las filas que estén completamente ocupadas.

2. Desarrollo

1. **Análisis del problema y consideraciones de la solución**

   El tablero se representa como un arreglo dinámico de `unsigned char`. Cada celda del tablero ocupa exactamente 1 bit, lo que minimiza el uso de memoria. Un tablero de 16×24 celdas ocupa solo $16 \times 24/8 = 48$ bytes, frente a los 384 bytes que consumiría un arreglo de bool.

   Por su parte, cada pieza se define dentro de una cuadrícula de 4×4, ocupando exactamente 16 bits. El estado de cada celda se determina mediante la posición calculada como fila $\times$ 4 + columna. Esta estructura permite que la pieza completa se almacene directamente en un registro de la CPU, eliminando la necesidad de gestionar memoria en el heap durante las operaciones de juego

   La lógica de rotación se ejecuta mediante la función rotarPieza(), la cual realiza un giro de 90° en sentido horario aplicando la transformación geométrica $(fila, col) \rightarrow (col, 2 - fila)$. El proceso se lleva a cabo iterando sobre los 16 bits de la pieza y construyendo un nuevo valor de tipo unsigned short en una variable local. Al emplear exclusivamente aritmética de bits, se evita usar arreglos auxiliares.

2. **Esquema de tareas de los algoritmos**

   El desarrollo se organizó en tres módulos independientes, cada uno con responsabilidades delimitadas:

Módulo	Archivo	Responsabilidades
Tablero	tablero.cpp / .h	Creación/destrucción del tablero con `new`/`delete`; acceso bit a bit (`estaOcupado`, `ocuparCelda`, `limpiarCelda`); detección y eliminación de filas completas.
Pieza	pieza.cpp / .h	Representación en 16 bits; lectura de bits; rotación sin memoria adicional; generación aleatoria de 7 tipos; validación de posición; fijado al tablero.
Principal	main.cpp	Flujo del juego: inicialización, bucle de entrada de comandos, llamada a funciones de tablero y pieza, detección de game over y liberación de memoria.

Tabla 1: Módulos del proyecto

El software se organiza en funciones modulares que separan la lógica de control de la manipulación de datos a bajo nivel. Estas tareas incluyen la gestión del tablero mediante bits, la rotación geométrica de las piezas y la verificación de colisiones, asegurando una ejecución eficiente y un uso mínimo de recursos de memoria.

Función	Tarea principal
`crearTablero()`	Calcular `bytesPorFila` = ancho/8; reservar arreglo dinámico con `new`; inicializar en cero.
`destruirTablero()`	Liberar memoria con `delete[]`; poner puntero en `nullptr` para evitar doble liberación.
`estaOcupado()`	Calcular índice de byte y posición de bit; aplicar máscara con AND.
`ocuparCelda()`	Calcular índice y bit; aplicar máscara con OR (encender bit).
`limpiarCelda()`	Calcular índice y bit; aplicar máscara con AND y NOT (apagar bit).
`filaLlena()`	Verificar que todos los bytes de la fila sean `0xFF` (255).
`eliminarFila()`	Copiar cada fila hacia abajo (`memmove` byte a byte); limpiar fila 0.
`limpiarFilasCompletas()`	Recorrer filas; si llena, eliminar y decrementar índice para re-evaluar.
`bitPiezaEncendido()`	Calcular índice lineal en cuadrícula 4×4; aplicar máscara con AND.
`rotarPieza()`	Transformar $(f, c) \rightarrow (c, 2-f)$ para cada bit encendido; construir nuevo `unsigned short`.
`generarPiezaAleatoria()`	Usar `rand()%7`; encender bits específicos según la forma clásica de cada pieza.
`piezaCabe()`	Para cada bit activo, calcular posición real; verificar límites y colisión con tablero.
`fijarPieza()`	Para cada bit activo, llamar a `ocuparCelda()` con la posición real.

Tabla 2: Funciones del proyecto

3. **Algoritmos Implementados**

- **Representación del Tablero con Bits Empaquetados:** El tablero almacena cada celda como un único bit dentro de un arreglo de unsigned char. El acceso a una celda específica $(fila, columna)$ se resuelve mediante un esquema de indexación que descompone la ubicación en un índice de byte y un desplazamiento de bit (offset) dentro de dicho byte:

```
// Localizacion del byte y bit especifico
int byteEnFila = columna / 8;
int bitEnByte  = columna % 8;
// Calculo del indice lineal en el arreglo dinamico
int indice     = fila * bytesPorFila + byteEnFila;
// Mascara para operacion bitwise (lectura/escritura)
unsigned char mascara = 1 << bitEnByte;
```

Una vez calculados el índice y la máscara, la manipulación de la celda se realiza mediante operadores lógicos a nivel de bits. Este enfoque permite leer, activar o borrar el estado de una celda sin alterar la información de los bits adyacentes dentro del mismo byte:

```
// 1. Lectura (Evalua si el bit esta en 1)
(celdas[indice] & mascara) != 0
// 2. Escritura (Pone el bit en 1 sin alterar los demas)
celdas[indice] |= mascara
// 3. Borrado (Pone el bit en 0 usando el complemento a uno)
celdas[indice] &= ~mascara
```

- **Rotación de Pieza:** La rotación de 90° en sentido horario implementado en la función `rotarPieza()` se fundamenta en una transformación lineal sobre una cuadrícula de $N \times N$. Para una matriz de 4×4 ($N = 4$), la regla de mapeo para trasladar una celda activa desde su posición original $(fila, columna)$ a su nueva posición es:

$$(fila, columna) \rightarrow (columna, 2 - fila)$$

  El algoritmo evalúa secuencialmente los 16 bits del registro de la pieza original. Si un bit está activo, se calcula su nueva posición espacial y se proyecta mediante un desplazamiento de bits en una variable local de tipo unsigned short.

- **Detección y Eliminación de Filas Completas:** La rutina principal recorre verticalmente el tablero de arriba hacia abajo evaluando cada fila. Su diseño integra una corrección de índice para el comportamiento del juego: cuando una fila es eliminada y las filas superiores bajan para ocupar su lugar, el algoritmo decrementa el iterador para reevaluar la misma posición espacial. Este mecanismo previene el "salto de filasçuando se completan múltiples líneas consecutivas.

```
for (int fila = 0; fila < alto; fila++) {
    if (filaLlena(celdas, bytesPorFila, fila)) {
        eliminarFila(celdas, bytesPorFila, fila);
        fila--;  // Re-evaluar el mismo indice (la fila de arriba bajo)
    }
}
```

  Se valida el estado de la memoria usando la función filaLlena(), dado que cada celda del tablero se representa con un bit, una fila de Tetris se considera llena si todos los bits que la componen están activos. El algoritmo evalua bytes completos (bloques de 8 bits), si el valor del byte es exactamente 255 ($0xFF$ en hexadecimal), significa que las 8 celdas están ocupadas simultáneamente.

- **Validación de Posición:** Antes de cualquier movimiento se verifica que cada celda activa de la pieza no colisione con los límites del tablero ni con celdas ya ocupadas. La función bool piezaCabe() regresa false en el primer conflicto.

  La función trabaja con dos sistemas de coordenadas distintos que se combinan para obtener la posición real de cada celda:

- **Coordenadas internas (filaInterna, colInterna):** La posición de una celda dentro de la cuadrícula 4×4 propia de la pieza. Siempre toman valores entre 0 y 3, independientemente de dónde esté la pieza en el tablero.
- **Posición de anclaje (filaInterna, colInterna):** Las coordenadas de la esquina superior izquierda de la cuadrícula 4×4 dentro del tablero. Es el valor que cambia cuando el jugador mueve la pieza.
- **Coordenadas reales (filaReal, colReal):** La posición efectiva de cada celda activa dentro del tablero, obtenida sumando ambos sistemas:

```
filaReal = filaPieza + filaInterna
colReal  = colPieza  + colInterna
```

- **Gestión de Memoria Dinámica:** El tablero es el único objeto que usa el heap. Se crea con new[] y se destruye con delete[]. El puntero se pasa por referencia (unsigned char*&) para que las funciones modifiquen el puntero original y puedan anularlo tras la liberación.

  El parámetro celdas se declara como unsigned char*&, es decir, una referencia a un puntero. Para entender por qué es necesario, hay que tener claro qué hace cada parte, unsigned char* es un puntero, una variable que guarda una dirección de memoria y & convierte ese puntero en una referencia, lo que significa que la función no recibe una copia del puntero sino el puntero original del llamador.

4. **Problemas de Desarrollo Afrontados** A continuación se presentan los problemas que se presentaron durante el desarrollo de la solución y la forma en que se resolvieron.

   - **Filas que No Se Eliminan Correctamente:** Tras completar varias filas simultáneas, algunas permanecen y la pila de piezas se eleva incorrectamente. Al incrementar fila después de eliminar una fila hace que el bucle salte la fila que acaba de descender. Se opto por decrementar el iterador (fila–) al final del if (filaLlena()) para que el iterador evalúe el mismo índice en la siguiente iteración (la fila que bajó).
   - **Rotación que Saca la Pieza de los Límites:** Al rotar cerca de un borde, la pieza aparece fuera del tablero o la rotación se rechaza siempre, como la cuadrícula 4×4 puede tener el centro de masa en posiciones distintas según la orientación, rotar cambia el offset efectivo. Se debe calcular la pieza rotada y pasarla a piezaCabe() antes de aceptarla. Si no cabe, se desecha piezaRotada y se conserva piezaActual.
   - **Desplazamiento de rotación:** Para reducir el rechazo de piezas por desplazamientos no deseados, se recalibró el eje de giro de la transformación. Al cambiar la proyección original de $(fila, col) \to (col, 3 - fila)$ a $(fila, col) \to (col, 2 - fila)$, se reduce el límite de la columna nueva al rotar.

5. **Evolución de la Solución**

Etapa	Estado del Programa	Decisión de Diseño Clave
1. Tablero	Solo tablero estático con bits; impresión en consola.	Elegir `unsigned char[]` sobre `bool[]` o `int[]`; definir `bytesPorFila` = ancho/8.
2. Pieza	Representar una pieza fija; dibujarla sobre el tablero.	Codificar pieza como `unsigned short` 16 bits; convención bit 0 = (0,0).
3. Rotación	Implementar rotación sin arrays auxiliares.	Transformación $(f, c) \rightarrow (c, 3-f)$; retorno por valor sin `new`.
4. Movimiento	Mover la pieza con colisión de bordes.	`piezaCabe()` genérico: funciona para mover, rotar y fijar.
5. Fijado	Fijar pieza al tablero al no poder bajar.	`fijarPieza()` reutiliza `ocuparCelda()`; modularidad completa.
6. Game Over	Detectar derrota al generar pieza sin espacio.	`piezaCabe()` también valida la posición inicial (fila 0).

Tabla 3: Etapas de desarrollo del proyecto

6. **Consideraciones de Implementación**

- **Punteros por Referencia en Funciones del Tablero:** Las funciones que deben modificar el puntero celdas o la variable bytesPorFila del llamador reciben estos argumentos como referencias: unsigned char*& celdas, int& bytesPorFila. Sin el &, la función trabajaría con una copia local y el tablero original quedaría sin crear o sin liberar.
- **El "fila–.en limpiarFilasCompletas es Crítico:** Si se elimina esa línea, tras eliminar una fila el contador avanza y nunca evalúa la fila que bajó. Con múltiples filas completas consecutivas solo se eliminaría una de cada dos.
- **Inicialización Explícita a Cero del Tablero:** Aunque en muchos sistemas new puede devolver memoria con ceros, el estándar de C++ no lo garantiza para new[] sobre tipos primitivos. El bucle de inicialización en crearTablero() asegura que ninguna celda esté marcada como ocupada al inicio del juego.
- **Centralización de la Lógica de Bits:** Toda lectura de bits pasa por bitPiezaEncendido() y toda escritura por ocuparCelda() / limpiarCelda(). Esto permite cambiar la convención de bits en un solo lugar sin alterar el resto del código.

- **Diferencias Entre el Pre-Informe y la Implementación Final:** El pre-informe describió un diseño inicial que evolucionó durante la implementación. Las siguientes decisiones de rediseño fueron conscientes y mejoraron la solución:

Aspecto	Pre-informe (planeado)	Implementación final (por qué es mejor)
Estructura del tablero	`unsigned char**` doble puntero, reserva fila por fila con ciclo de `new[]`.	`unsigned char*` arreglo plano único. Un solo `new[]`, un solo `delete[]`. Menos riesgo de memory leak y menor overhead de punteros.
Módulos	Cuatro módulos: `main`, tablero, pieza y juego (`juego.h/cpp` separado).	Tres módulos. La lógica del juego es suficientemente simple para vivir en `main.cpp` sin justificar un módulo extra.
Rotación	Sub-estructura temporal en el tablero con reemplazo directo.	`unsigned short` local con transformación $(f, c) \to (c, 3-f)$. Cumple la restricción de no usar memoria adicional; costo $O(1)$.
Detección de colisión	Operación pieza & tablero por filas (máscaras «`,` »).	`piezaCabe()` evalúa cada bit activo individualmente. Más preciso, funciona con piezas en cualquier posición.
Ingreso de piezas	Zona superior invisible para ingreso gradual.	Pieza en fila 0; `piezaCabe()` valida antes de mostrar. Mismo resultado, menor complejidad.

Tabla 4: Comparación entre diseño planeado e implementación final

- **Resumen de Eficiencia:**

Operación	Complejidad Temporal	Complejidad Espacial
`estaOcupado` / `ocuparCelda` / `limpiarCelda`	$O(1)$	$O(1)$ — sin memoria extra
`bitPiezaEncendido`	$O(1)$	$O(1)$
`rotarPieza`	$O(16) = O(1)$	$O(1)$ — solo variable local
`piezaCabe`	$O(16) = O(1)$	$O(1)$
`fijarPieza`	$O(16) = O(1)$	$O(1)$
`filaLlena`	$O(\text{bytesPorFila})$	$O(1)$
`eliminarFila`	$O(\text{alto} \times \text{bytesPorFila})$	$O(1)$
`limpiarFilasCompletas`	$O(\text{alto}^2 \times \text{bytesPorFila})$	$O(1)$
`crearTablero`	$O(\text{alto} \times \text{bytesPorFila})$	$O(\text{alto} \times \text{bytesPorFila})$
Memoria total del tablero	—	ancho $\times$ alto/8 bytes

Tabla 5: Complejidad temporal y espacial de las operaciones

3. Conclusiones

La implementación demuestra tres principios fundamentales del desafío:

1. **Memoria dinámica precisa:** el tablero se reserva con new[], se inicializa explícitamente y se libera con delete[] en todas las rutas de salida, con el puntero anulado para evitar punteros colgantes.

2. **Eficiencia por representación de bits:** usar 1 bit por celda reduce el consumo de memoria hasta 32 veces frente a int[] y 8 veces frente a bool[], sin perder accesibilidad gracias a las funciones de acceso con máscaras.

3. **Rotación sin memoria dinámica:** la transformación geométrica $(fila, col) \to (col, 2 - fila)$ calcula la pieza rotada como un entero local en la pila, cumpliendo la restricción del desafío de no almacenar rotaciones en el heap.

La separación en módulos (tablero.cpp, pieza.cpp, main.cpp) facilita el mantenimiento y la verificación independiente de cada componente, aplicando el principio de responsabilidad única en un contexto de programación de bajo nivel en C++.